

Trần Quang Trường - MSSV: 24521901
February 23, 2025

Thuật toán sắp xếp (Sorting Algorithms)

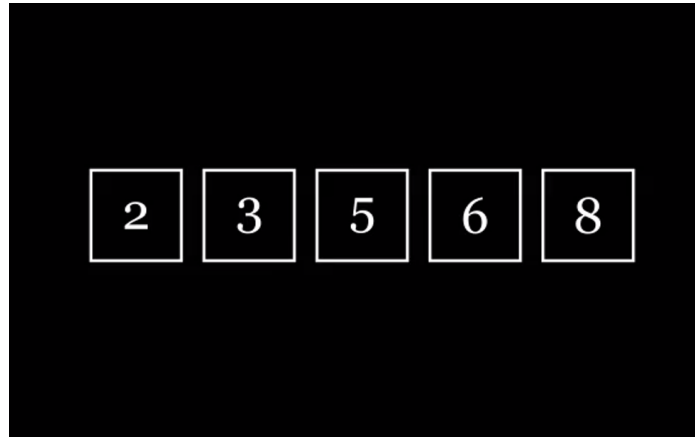
Contents

1	Giới thiệu	2
2	Một số thuật toán sắp xếp	2
2.1	Interchange Sort	2
2.1.1	Quick Sort (Sắp xếp nhanh)	4
2.1.2	Bubble Sort (Sắp xếp nổi bọt)	5
2.2	Insertion Sort	9
2.2.1	Binary Insertion Sort (Sắp xếp chèn nhị phân)	10
2.2.2	Shell Sort (Sắp xếp theo khoảng)	11
2.3	Selection Sort	13
2.3.1	Heap Sort (Sắp xếp vun đống)	14
2.4	Merge Sort	15
2.4.1	Natural Merge Sort (Sắp xếp trộn tự nhiên)	17
2.4.2	K-way Merge Sort (Sắp xếp trộn K chiều)	19
2.5	Specific	21
2.5.1	Counting Sort (Sắp xếp đếm)	21
2.5.2	Radix Sort (Sắp xếp cơ số)	23
3	Tổng kết	24

1 Giới thiệu

Sorting hay sắp xếp là một trong những bài toán cơ bản và quan trọng nhất. Từ việc tổ chức dữ liệu, sắp xếp danh sách công việc đến cải thiện tốc độ tìm kiếm, các thuật toán sắp xếp thường có mặt ở khắp mọi nơi.

Về bản chất, thuật toán sắp xếp là quá trình sắp xếp một tập hợp các phần tử theo một thứ tự nhất định, thường là tăng dần hoặc giảm dần. Có nhiều thuật toán sắp xếp khác nhau, mỗi thuật toán có ưu điểm, nhược điểm riêng và phù hợp với từng trường hợp cụ thể.



Hình minh họa một tập hợp được sắp xếp theo thứ tự tăng dần

Trong bài viết này, chúng ta sẽ cùng tìm hiểu về các thuật toán sắp xếp phổ biến, cách thức mà chúng hoạt động, và phân tích độ phức tạp của từng thuật toán cụ thể. Qua đó sẽ giúp chúng ta hiểu rõ hơn về cách mà thuật toán được hình thành và cách lựa chọn thuật toán phù hợp nhất cho từng bài toán thực tế.

2 Một số thuật toán sắp xếp

Dựa vào từng thao tác cơ bản dùng để xây dựng thuật toán, ta có thể phân loại các thuật toán sắp xếp thành 5 dạng chính: Thao tác hoán vị (Interchange), Thao tác chèn (Insertion), Thao tác chọn (Selection), Thao tác trộn (Merge), và những thuật toán không dựa vào các thao tác cơ bản đã nhắc tới (Specific).

2.1 Interchange Sort

Interchange Sort hay **sắp xếp hoán vị** là một thuật toán sắp xếp đơn giản, hoạt động bằng cách đổi chỗ vị trí các cặp nghịch thế¹ trong mảng.

¹Cặp nghịch thế là cặp phần tử đứng sai thứ tự theo yêu cầu sắp xếp. Cụ thể hơn, cho mảng $a_1, a_2, \dots, a_n$ với yêu cầu sắp xếp tăng dần thì (a_i, a_j) là một cặp nghịch thế khi và chỉ khi:

$$i < j \quad \text{và} \quad a_i > a_j$$

Ý tưởng thuật toán:

1. Duyệt tuần tự các phần tử trong mảng.
2. Lần lượt xét phần tử hiện tại với các phần tử nằm sau nó.
3. Nếu là cặp nghịch thế thì đổi chỗ hai phần tử đó.
4. Lặp lại cho đến khi toàn bộ mảng được sắp xếp.

Chi tiết cài đặt:

```
#include <bits/stdc++.h>

int n = 5;
int a[] = {0, 3, 2, 8, 6, 5};

int main() {
    for(int i = 1; i <= n; ++i) {
        for(int j = i + 1; j <= n; ++j) {
            // Kiểm tra (a[i], a[j]) có phải cặp nghịch thế hay không?
            if(a[i] > a[j]) {
                // Đổi chỗ nếu (a[i], a[j]) là cặp nghịch thế
                std::swap(a[i], a[j]);
            }
        }
    }

    // In ra mảng sau khi được sắp xếp tăng dần
    for(int i = 1; i <= n; ++i) {
        std::cout << a[i] << " ";
    }
}
```

Độ phức tạp:

- **Thời gian:**

- Với hai vòng lặp lồng nhau và không có lệnh thoát vòng lặp sớm (break), ta đạt độ phức tạp thuật toán ở mọi trường hợp đều là $O(n^2)$.

- **Không gian:**

- Vì thuật toán chỉ thao tác trực tiếp trên mảng đầu vào mà không cần thêm mảng phụ. Nên độ phức tạp không gian của Interchange Sort là $O(1)$.

2.1.1 Quick Sort (Sắp xếp nhanh)

Ta thấy rằng thuật toán Interchange Sort sẽ chạy không hiệu quả với những giới hạn lớn bởi vì thuật toán thực hiện một số phép đổi chỗ cặp nghịch thế không cần thiết. Lấy ví dụ với mảng $a = \{2, 4, 3, 1\}$. Với thuật toán Interchange sort, ta cần 4 thao tác đổi chỗ các cặp lần lượt là (a_1, a_4) , (a_2, a_3) , (a_2, a_4) và (a_3, a_4) . Trong khi, thực tế ta chỉ cần 2 thao tác lần lượt là (a_1, a_4) và (a_2, a_4) để đưa mảng về dạng được sắp xếp. Cải tiến từ Interchange sort, **Quick Sort** hay **sắp xếp nhanh** áp dụng kĩ thuật chia để trị² để giảm thiểu số thao tác đổi chỗ không cần thiết của thuật toán gốc.

Ý tưởng thuật toán:

1. Chọn một phần tử trong mảng làm pivot (thường là phần tử đầu, cuối, giữa hoặc ngẫu nhiên).
2. Chia mảng thành 2 phần sao cho:
 - a) Các phần tử nhỏ hơn pivot nằm phần bên trái.
 - b) Các phần tử lớn hơn hoặc bằng pivot nằm phần bên phải.
3. Gọi Quick Sort trên hai phần mảng đã chia.

Khi gọi hàm đệ quy, ta sẽ chia mảng thành 2 phần: **phần bên phải** gồm các phần tử nhỏ hơn pivot và **phần bên trái** gồm các phần tử lớn hơn hoặc bằng pivot, ta sẽ đảm bảo một điều rằng **phần tử lớn nhất** của mảng bên trái sẽ **luôn bé hơn phần tử nhỏ nhất** của mảng bên phải. Do đó, nếu ta tiếp tục gọi hàm đệ quy để sắp xếp lại theo thứ tự tăng dần trên 2 mảng này, thì mảng gồm cả hai phần đó cũng được sắp xếp tăng dần.

Chi tiết cài đặt:

```
#include <bits/stdc++.h>

int n = 5;
int a[] = {0, 2, 1, 4, 3, 5};

// Gọi hàm đệ quy sắp xếp các phần tử trong đoạn [l, r]
void QuickSort(int l, int r) {
    // Nếu mảng chỉ còn một phần tử hoặc rỗng thì dừng lại
    if(l >= r) return;

    // Lấy pivot là phần tử cuối
    int pivot = a[r];
    int ptr = l;
    for(int i = l; i <= r - 1; ++i) {
        if(a[i] < pivot) {
```

²Chia để trị hay trong tiếng anh là Divide And Conquer, là một kĩ thuật chia bài toán cần giải quyết thành các bài toán nhỏ.

```

        // Đưa phần tử có giá trị bé hơn pivot vào phần bên trái
        std::swap(a[ptr++], a[i]);
    }
}
std::swap(a[ptr], a[r]);
QuickSort(l, ptr - 1);
QuickSort(ptr + 1, r);
}

int main() {
    // Gọi hàm đệ quy
    QuickSort(1, n);

    // In ra mảng sau khi được sắp xếp tăng dần
    for(int i = 1; i <= n; ++i) {
        std::cout << a[i] << " ";
    }
}

```

Độ phức tạp:

Thuật toán QuickSort phụ thuộc rất nhiều vào việc chọn pivot.

- **Thời gian:**

- **Trường hợp ngẫu nhiên và tốt nhất:** Khi pivot được chọn tốt, mảng được chia thành hai phần tương đối bằng nhau. Khi đó, số lần gọi đệ quy tỷ lệ với số lần chia đôi mảng, tức là $\log n$. Mỗi lần gọi hàm đệ quy thực hiện $O(n)$ phép so sánh, nên tổng thời gian chạy là $O(n \log n)$.
- **Trường hợp tệ nhất:** Nếu pivot được chọn không tối ưu (phần tử nhỏ nhất hoặc lớn nhất), mỗi lần chia mảng chỉ giảm đi 1 phần tử. Khi đó, thuật toán gọi đệ quy n lần, mỗi lần mất $O(n)$, dẫn đến độ phức tạp thời gian là $O(n^2)$.

- **Không gian:**

- **Trường hợp ngẫu nhiên và tốt nhất:** Ta cần $O(\log n)$ không gian cho stack do độ sâu của lời gọi đệ quy.
- **Trường hợp tệ nhất:** Ta cần $O(n)$ không gian cho stack do độ sâu của lời gọi đệ quy.

2.1.2 Bubble Sort (Sắp xếp nổi bọt)

Gác lại những bước xử lý phức tạp của thuật toán Quick Sort, bây giờ ta sẽ cùng tìm hiểu về một thuật toán cũng đơn giản không kém với Interchange Sort. Thuật toán ta nói tới là **Bubble Sort** hay **sắp xếp nổi bọt**, một thuật toán sắp xếp dựa trên nguyên lý nổi lên của bong bóng trong nước.

Ý tưởng thuật toán:

- Ở một lượt duyệt, ta sẽ đưa dần phần tử lớn nhất "nổi" lên cuối mảng, bao gồm các bước:
 - Duyệt qua mảng từ đầu đến cuối.
 - So sánh từng cặp phần tử liên tiếp.
 - Nếu phần tử trước lớn hơn phần tử sau $\rightarrow$ đổi chỗ.
- Lặp lại quá trình trên với phần còn lại của mảng (không tính phần đã sắp xếp).
- Tiếp tục cho đến khi không còn cặp nào cần đổi chỗ.

Cài đặt chi tiết:

```
#include <bits/stdc++.h>

int n = 5;
int a[] = {0, 2, 1, 4, 3, 5};

int main() {
    bool swapped = false;
    // Duyệt các phần tử trong mảng
    for(int i = 1; i <= n; ++i) {
        swapped = false;
        // Đưa phần tử lớn nhất phần còn lại của mảng "nổi" lên cuối mảng
        for(int j = 1; j <= n - i; ++j) {
            // Nếu a[j] > a[j + 1] đưa đổi chỗ a[j], a[j + 1]
            if(a[j] > a[j + 1]) {
                std::swap(a[j], a[j + 1]);
                // Đánh dấu vẫn mảng vẫn chưa được sắp xếp xong
                swapped = true;
            }
        }
        // Nếu như mảng đã được sắp xếp xong thì kết thúc bài toán
        if(!swapped) break;
    }

    // In ra mảng sau khi được sắp xếp tăng dần
    for(int i = 1; i <= n; ++i) {
        std::cout << a[i] << " ";
    }
}
```

Độ phức tạp:

- Thời gian:**
 - **Trường hợp tốt nhất:** Nếu mảng đầu vào đã được sắp xếp, thuật toán chỉ cần duyệt qua mảng một lần để kiểm tra. Khi đó, ta có thể dừng sớm sau lần duyệt đầu tiên, dẫn đến độ phức tạp thời gian là $O(n)$.

- **Trường hợp ngẫu nhiên và tệ nhất:** Nếu mảng được sắp xếp theo thứ tự giảm dần, thuật toán phải thực hiện đầy đủ n lần duyệt để đưa mảng về trạng thái tăng dần. Khi đó, mỗi lần duyệt có thể thực hiện đến $O(n)$ phép hoán đổi, dẫn đến tổng độ phức tạp thời gian là $O(n^2)$.

- **Không gian:**

- Vì chỉ sử dụng một biến tạm để hoán đổi các phần tử trong mảng nên độ phức tạp không gian của thuật toán là $O(1)$.

Shake Sort (Cải tiến của thuật toán Bubble Sort)

Cùng nhắc lại ý tưởng của thuật toán, Bubble Sort, ý tưởng thuật toán sẽ là đi từ đầu tới cuối mảng và mang theo phần tử lớn nhất theo, sau đó lại quay lại từ vị trí đầu để đi tìm phần tử lớn nhất tiếp theo mang về cuối. Thay vì thế tại sao lúc đưa xong phần tử lớn nhất về đúng vị trí của nó, ta không đi về và mang theo phần tử bé nhất để đưa về đầu mảng? **Shake Sort** hay **sắp xếp lắc** dựa trên ý tưởng trên, một lượt duyệt trong Shake Sort sẽ vừa đưa phần tử lớn nhất hiện tại vào cuối mảng và vừa đưa phần tử bé nhất hiện tại về đầu mảng.

Ý tưởng thuật toán:

1. Ở một lượt duyệt, ta sẽ đưa dần phần tử lớn nhất "nổi" lên cuối mảng sau đó đưa phần tử bé nhất "chìm" xuống đầu mảng, bao gồm các bước:
 - a) Duyệt qua mảng từ đầu đến cuối.
 - b) So sánh từng cặp phần tử liên tiếp.
 - c) Nếu phần tử trước lớn hơn phần tử sau $\rightarrow$ đổi chỗ.
 - d) Duyệt qua mảng từ cuối về đầu.
 - e) So sánh từng cặp phần tử liên tiếp.
 - f) Nếu phần tử trước lớn hơn phần tử sau $\rightarrow$ đổi chỗ.
2. Lặp lại quá trình trên với phần còn lại của mảng (không tính phần đã sắp xếp).
3. Tiếp tục cho đến khi không còn cặp nào cần đổi chỗ.

Chi tiết cài đặt:

```
#include <bits/stdc++.h>

int n = 5;
int a[] = {0, 2, 1, 4, 3, 5};

int main() {
    for(int l = 1, r = n; l < r; ++l, --r) {
        // Đoạn [l, r] đại diện cho một đoạn mảng chưa được sắp xếp

        bool swapped = false;
```

```

// Đưa phần tử lớn nhất phần còn lại của mảng "nổi" lên cuối mảng
for(int i = 1; i <= r - 1; ++i) {
    // Nếu a[i] > a[i + 1] đưa đổi chỗ a[i], a[i + 1].
    if(a[i] > a[i + 1]) {
        std::swap(a[i], a[i + 1]);
        // Đánh dấu vẫn mảng vẫn chưa được sắp xếp xong
        swapped = true;
    }
}
// Đưa phần tử bé nhất phần còn lại của mảng "chìm" về đầu mảng
for(int i = r - 1; i >= l + 1; --i) {
    // Nếu a[i - 1] < a[i] đưa đổi chỗ a[i - 1], a[i].
    if(a[i - 1] > a[i]) {
        std::swap(a[i - 1], a[i]);
    }
}
// Nếu như mảng đã được sắp xếp xong thì kết thúc bài toán
if(!swapped) break;
}
// In ra mảng sau khi được sắp xếp tăng dần
for(int i = 1; i <= n; ++i) {
    std::cout << a[i] << " ";
}
}

```

Độ phức tạp:

- **Thời gian:**

- **Trường hợp tốt nhất:** Nếu mảng đầu vào đã được sắp xếp, thuật toán chỉ cần duyệt qua mảng một lần để kiểm tra. Khi đó, ta có thể dừng sớm sau lần duyệt đầu tiên, dẫn đến độ phức tạp thời gian là $O(n)$.
- **Trường hợp ngẫu nhiên và tệ nhất:** Nếu mảng được sắp xếp theo thứ tự giảm dần, thuật toán phải thực hiện n lần duyệt qua mảng theo cả hai chiều để đưa mảng về trạng thái tăng dần. Khi đó, mỗi lần duyệt có thể thực hiện đến $O(n)$ phép hoán đổi, dẫn đến tổng độ phức tạp thời gian là $O(n^2)$.

- **Không gian:**

- Vì chỉ sử dụng một biến tạm để hoán đổi các phần tử trong mảng nên độ phức tạp không gian của thuật toán là $O(1)$.

Dù ý tưởng của Shake Sort khá hay về mặt lý thuyết; nhưng trong thực tế, Shake Sort không hề đem lại hiệu quả tốt hơn về mặt thời gian mà còn khiến việc cài đặt trở nên khó khăn hơn. Dù được gọi là thuật toán cải tiến từ Bubble Sort, nhưng Shake Sort lại được coi là một cải tiến lỗi không hơn không kém.

2.2 Insertion Sort

Insertion Sort hay **sắp xếp chèn** là một thuật toán sắp xếp đơn giản, hoạt động bằng cách chèn từng phần tử vào vị trí thích hợp trong dãy đã được sắp xếp.

Ý tưởng thuật toán:

1. Duyệt tuần tự từng phần tử trong mảng, bắt đầu từ phần tử thứ hai.
2. Lấy phần tử hiện tại và so sánh với các phần tử trước nó trong dãy đã sắp xếp.
3. Dịch chuyển các phần tử lớn hơn phần tử đang xét sang phải để tạo vị trí thích hợp.
4. Chèn phần tử đang xét vào đúng vị trí để đảm bảo thứ tự tăng dần.
5. Lặp lại cho đến khi toàn bộ mảng được sắp xếp.

Chi tiết cài đặt:

```
#include <bits/stdc++.h>

int n = 5;
int a[] = {0, 3, 2, 8, 6, 5};

int main() {
    for (int i = 2; i <= n; ++i) {
        int key = a[i], j = i - 1;
        // Dịch chuyển các phần tử lớn hơn key sang phải
        while (j >= 1 && a[j] > key) {
            a[j + 1] = a[j];
            --j;
        }
        // Chèn phần tử key vào đúng vị trí
        a[j + 1] = key;
    }

    // In ra mảng sau khi được sắp xếp tăng dần
    for (int i = 1; i <= n; ++i) {
        std::cout << a[i] << " ";
    }
}
```

Độ phức tạp:

- **Thời gian:**
 - Trường hợp tốt nhất (mảng đã sắp xếp): $O(n)$ vì mỗi phần tử chỉ cần so sánh một lần.

- Trường hợp trung bình và xấu nhất: $O(n^2)$ do mỗi phần tử có thể cần dịch chuyển gần như toàn bộ mảng phía trước nó.

- **Không gian:**

- Insertion Sort là thuật toán sắp xếp tại chỗ (in-place sort), chỉ sử dụng một biến tạm thời (key) để lưu giá trị cần chèn.
- Do đó, độ phức tạp về không gian là $O(1)$.

2.2.1 Binary Insertion Sort (Sắp xếp chèn nhị phân)

Insertion Sort là thuật toán sắp xếp chèn trực tiếp, hoạt động bằng cách lấy từng phần tử và chèn nó vào vị trí thích hợp trong phần đã được sắp xếp của mảng. Thông thường, ta sẽ sử dụng vòng lặp để tìm vị trí cần chèn với độ phức tạp $O(n)$. Tuy nhiên, nếu sử dụng **tìm kiếm nhị phân**³, ta có thể giảm độ phức tạp của bước này xuống $O(\log n)$, từ đó cải thiện thời gian chạy của thuật toán. Từ đó ta có thuật toán **Binary Insertion Sort** hay **sắp xếp chèn nhị phân**.

Ý tưởng thuật toán:

1. Duyệt từng phần tử trong mảng (bắt đầu từ phần tử thứ hai).
2. Dùng **tìm kiếm nhị phân** để tìm vị trí thích hợp trong phần đã sắp xếp.
3. Dịch chuyển các phần tử để tạo khoảng trống cho phần tử cần chèn.
4. Chèn phần tử vào vị trí tìm được.
5. Lặp lại quá trình trên cho đến khi toàn bộ mảng được sắp xếp.

Chi tiết cài đặt:

```
#include <bits/stdc++.h>

int n = 5;
int a[] = {0, 5, 3, 2, 4, 1};

int main() {
    for (int i = 2; i <= n; ++i) {
        int key = a[i];
        int left = 1, right = i - 1;

        // Tìm vị trí chèn bằng tìm kiếm nhị phân
        while (left <= right) {
            int mid = (left + right) / 2;
            if (a[mid] > key)
                right = mid - 1;
        }
    }
}
```

³Tìm kiếm nhị phân hay Binary Search là một thuật toán tìm kiếm xác định vị trí của một giá trị cần tìm trong một mảng đã được sắp xếp.

```

        else
            left = mid + 1;
    }

    // Dời các phần tử để tạo khoảng trống cho phần tử cần chèn
    for (int j = i; j > left; --j) {
        a[j] = a[j - 1];
    }

    // Chèn phần tử vào đúng vị trí
    a[left] = key;
}

// In ra mảng sau khi được sắp xếp tăng dần.
for (int i = 1; i <= n; ++i) {
    std::cout << a[i] << " ";
}
}

```

Độ phức tạp:

- **Thời gian:**
 - **Trường hợp tốt nhất:** Khi mảng đã được sắp xếp, ta chỉ cần thực hiện $O(\log n)$ phép so sánh mà không cần dời phần tử nào. → Độ phức tạp: $O(n \log n)$.
 - **Trường hợp trung bình và tệ nhất:** Khi mảng bị xáo trộn hoàn toàn hoặc sắp xếp theo thứ tự giảm dần, mỗi phần tử phải tìm vị trí bằng tìm kiếm nhị phân và dịch chuyển nhiều phần tử. → Độ phức tạp: $O(n^2)$.
- **Không gian:**
 - Thuật toán chỉ sử dụng một vài biến tạm, không sử dụng mảng phụ. Do đó, độ phức tạp không gian của thuật toán là $O(1)$.

Dù Binary Insertion Sort giúp giảm số lần so sánh từ $O(n)$ xuống $O(\log n)$, nhưng vẫn phải thực hiện việc dịch chuyển phần tử, giữ độ phức tạp tổng thể ở mức $O(n^2)$. Vì thế, thuật toán này không mang lại lợi ích đáng kể so với Insertion Sort thông thường trong hầu hết các trường hợp thực tế.

2.2.2 Shell Sort (Sắp xếp theo khoảng)

Thuật toán Insertion Sort hoạt động hiệu quả khi mảng gần như đã được sắp xếp, nhưng khi áp dụng trên mảng có kích thước lớn và xáo trộn nhiều, thuật toán gặp vấn đề về tốc độ do phải dịch chuyển nhiều phần tử.

Cải tiến từ Insertion Sort, **Shell Sort** giúp giảm số lần dịch chuyển phần tử bằng cách sử dụng một dãy khoảng cách gap.

Ý tưởng thuật toán:

- Chia mảng thành các nhóm phần tử cách nhau một khoảng *gap*, sau đó sắp xếp từng nhóm bằng **Insertion Sort**.
- Giảm dần *gap* theo một quy tắc nhất định.
- Khi *gap* = 1, thực hiện Insertion Sort lần cuối trên toàn bộ mảng.

Nhờ việc sắp xếp trước các nhóm phần tử cách xa nhau, thuật toán giúp phần lớn phần tử tiến gần về đúng vị trí trước khi thực hiện Insertion Sort, giúp giảm số lần dịch chuyển và cải thiện hiệu suất trên mảng lớn.

Ý tưởng thuật toán:

1. Chọn một khoảng cách (*gap*) ban đầu, thường bằng $n/2$.
2. Chia mảng thành các nhóm nhỏ theo khoảng cách *gap*, sau đó sắp xếp từng nhóm bằng Insertion Sort.
3. Giảm dần khoảng cách (thường chia đôi *gap*) và tiếp tục quá trình trên.
4. Khi *gap* = 1, thực hiện Insertion Sort một lần cuối để hoàn tất việc sắp xếp.

Chi tiết cài đặt:

```
#include <bits/stdc++.h>

int n = 5;
int a[] = {0, 5, 3, 2, 4, 1};

int main() {
    // Bắt đầu với khoảng cách (gap) là n/2 và giảm dần
    for (int gap = n / 2; gap > 0; gap /= 2) {
        for (int i = gap + 1; i <= n; ++i) {
            int key = a[i];
            int j = i;

            // Dịch chuyển các phần tử theo khoảng cách gap
            while (j > gap && a[j - gap] > key) {
                a[j] = a[j - gap];
                j -= gap;
            }

            // Chèn phần tử vào đúng vị trí
            a[j] = key;
        }
    }

    // In ra mảng sau khi được sắp xếp tăng dần
    for (int i = 1; i <= n; ++i) {
```

```

        std::cout << a[i] << " ";
    }
}

```

Độ phức tạp:

- **Thời gian:** Độ phức tạp phụ thuộc vào cách chọn dãy gap.
 - **Trường hợp tốt nhất:** Nếu dãy gap được chọn phù hợp và mảng gần như đã sắp xếp, Shell Sort có thể đạt hiệu suất gần $O(n \log n)$.
 - **Trường hợp ngẫu nhiên:** Nếu sử dụng một dãy gap tốt như **dãy Knuth** ($h_k = 3h_{k-1} + 1$) hoặc **dãy Sedgewick**, độ phức tạp trung bình thường nằm trong khoảng $O(n^{1.25})$ đến $O(n \log^2 n)$.
 - **Trường hợp tệ nhất:** Nếu chọn dãy gap không tối ưu, thuật toán có thể có độ phức tạp lên tới $O(n^2)$.
- **Không gian:**
 - Thuật toán chỉ sử dụng một vài biến tạm, không sử dụng mảng phụ. Do đó, độ phức tạp không gian của thuật toán là $O(1)$.

Shell Sort là một thuật toán sắp xếp hiệu quả hơn Insertion Sort, đặc biệt là với các mảng lớn. Tuy nhiên, độ phức tạp của nó không ổn định và phụ thuộc vào cách chọn dãy gap, khiến nó ít phổ biến hơn so với các thuật toán như Merge Sort hay Quick Sort trong thực tế.

2.3 Selection Sort

Selection Sort hay **sắp xếp chọn** là một thuật toán sắp xếp đơn giản, hoạt động bằng cách tìm phần tử nhỏ nhất trong mảng và đặt nó vào vị trí đúng. Quá trình này lặp lại cho đến khi toàn bộ mảng được sắp xếp.

Ý tưởng thuật toán:

1. Duyệt qua từng vị trí trong mảng từ đầu đến cuối.
2. Tìm phần tử nhỏ nhất trong đoạn chưa sắp xếp.
3. Đổi chỗ phần tử nhỏ nhất với phần tử đầu tiên của đoạn chưa sắp xếp.
4. Lặp lại cho đến khi toàn bộ mảng được sắp xếp.

Chi tiết cài đặt:

```

#include <bits/stdc++.h>

int n = 5;
int a[] = {0, 5, 2, 8, 6, 3};

int main() {

```

```

for(int i = 1; i <= n; ++i) {
    int min_index = i;
    for(int j = i + 1; j <= n; ++j) {
        if(a[j] < a[min_index]) {
            min_index = j;
        }
    }
    std::swap(a[i], a[min_index]);
}

// In ra mảng sau khi được sắp xếp tăng dần.
for(int i = 1; i <= n; ++i) {
    std::cout << a[i] << " ";
}
}

```

Độ phức tạp:

- **Thời gian:**
 - Với hai vòng lặp lồng nhau và không có lệnh thoát vòng lặp sớm (break), ta đạt độ phức tạp thuật toán ở mọi trường hợp đều là $O(n^2)$.
- **Không gian:**
 - Thuật toán chỉ sử dụng một vài biến tạm, không sử dụng mảng phụ. Do đó, độ phức tạp không gian của thuật toán là $O(1)$.

2.3.1 Heap Sort (Sắp xếp vun đống)

Thuật toán Selection Sort không đạt hiệu quả tốt khi gặp những giới hạn lớn do phải tìm phần tử nhỏ nhất qua mỗi vòng lặp. **Heap Sort** hay sắp xếp vun đống tối ưu điều này bằng cách sử dụng **heap**⁴ để tìm phần tử lớn nhất/nhỏ nhất trong $O(\log n)$ thời gian thay vì $O(n)$.

Ý tưởng thuật toán:

1. Đưa tất cả phần tử của mảng vào một **max heap** (dùng `std::priority_queue`).
2. Liên tục lấy phần tử lớn nhất từ heap ra và đặt vào vị trí cuối mảng.
3. Kết thúc khi heap rỗng.

Chi tiết cài đặt:

⁴Heap là một cấu trúc dữ liệu dạng cây, dùng để thêm / xoá nhanh các phần tử, và xác định nhanh chóng phần tử lớn nhất/bé nhất hiện tại.

```

#include <bits/stdc++.h>

int n = 5;
int a[] = {0, 5, 2, 8, 6, 3};

int main() {
    std::priority_queue<int> pq;

    // Đưa toàn bộ phần tử vào max heap
    for (int i = 1; i <= n; ++i) {
        pq.push(a[i]);
    }

    // Lấy từng phần tử lớn nhất từ heap và đưa vào mảng
    for (int i = n; i >= 1; --i) {
        a[i] = pq.top();
        pq.pop();
    }

    // In ra mảng sau khi được sắp xếp tăng dần.
    for (int i = 1; i <= n; ++i) {
        std::cout << a[i] << " ";
    }
}

```

Độ phức tạp:

- **Thời gian:**

- Mỗi phần tử được chèn vào `std::priority_queue` mất $O(\log n)$. Vì có n phần tử, tổng thời gian để xây dựng heap là $O(n \log n)$.
 - Ta thực hiện n lần lấy phần tử lớn nhất từ heap, mỗi lần tốn $O(\log n)$. Do đó, tổng thời gian là $O(n \log n)$.
- Vì cả hai bước trên đều có độ phức tạp $O(n \log n)$, nên thuật toán Heap Sort có độ phức tạp thời gian là $O(n \log n)$ trong mọi trường hợp.

- **Không gian:**

- Thuật toán sử dụng `std::priority_queue`, tức là cần thêm $O(n)$ bộ nhớ để lưu trữ heap. Do đó, độ phức tạp không gian là $O(n)$.

2.4 Merge Sort

Merge Sort hay **sắp xếp trộn** là một thuật toán sắp xếp sử dụng phương pháp **chia để trị**, chia mảng thành hai nửa nhỏ hơn, sắp xếp từng nửa và sau đó trộn chúng lại để tạo thành mảng đã sắp xếp.

Ý tưởng thuật toán:

1. Nếu mảng chỉ có một phần tử, nó đã được sắp xếp.
2. Chia mảng thành hai nửa bằng nhau.
3. đệ quy sắp xếp từng nửa.
4. Trộn hai nửa đã sắp xếp lại để tạo thành mảng cuối cùng.

Chi tiết cài đặt:

```
#include <bits/stdc++.h>

int n = 5;
int a[] = {0, 5, 2, 8, 6, 3};

// Hàm trộn hai nửa đã sắp xếp của mảng
void merge(int l, int m, int r) {
    // Mảng tạm để trộn hai phần mảng
    int temp[r - l + 1];
    int i = l, j = m + 1, k = 0;

    // So sánh và trộn hai phần mảng
    while (i <= m && j <= r) {
        if (a[i] <= a[j]) temp[k++] = a[i++];
        else temp[k++] = a[j++];
    }

    // Sao chép phần còn lại nếu có
    while (i <= m) temp[k++] = a[i++];
    while (j <= r) temp[k++] = a[j++];

    // Sao chép các phần tử đã trộn vào mảng chính
    for (int t = 0; t < k; ++t) a[l + t] = temp[t];
}

// Hàm đệ quy thực hiện Merge Sort
void MergeSort(int l, int r) {
    // Nếu mảng chỉ còn một phần tử thì kết thúc.
    if(l == r) return;

    // Tìm vị trí chính giữa của mảng
    int m = (l + r) / 2;

    MergeSort(l, m);
    MergeSort(m + 1, r);
}
```



```

        // Trộn hai nửa đã sắp xếp
        merge(l, m, r);
    }

    int main() {
        // Gọi hàm đệ quy.
        MergeSort(1, n);

        // In ra mảng sau khi được sắp xếp tăng dần
        for(int i = 1; i <= n; ++i) {
            std::cout << a[i] << " ";
        }
    }
}

```

Độ phức tạp:

- **Thời gian:**
 - Mỗi lần chia mảng, ta chia đôi kích thước mảng $\log n$ lần.
 - Mỗi lần trộn hai mảng lại, ta mất $O(n)$ phép so sánh. Do đó, độ phức tạp thời gian là $O(n \log n)$ trong mọi trường hợp.
- **Không gian:**
 - Thuật toán sử dụng một mảng tạm có kích thước tối đa bằng kích thước của mảng gốc, nên độ phức tạp không gian là $O(n)$.

2.4.1 Natural Merge Sort (Sắp xếp trộn tự nhiên)

Thuật toán Merge Sort chuẩn chia mảng thành các phần bằng nhau ngay từ đầu mà không quan tâm đến tính chất đã sắp xếp của dữ liệu. Điều này đôi khi dẫn đến việc chia nhỏ không cần thiết. **Natural Merge Sort** hay sắp xếp trộn tự nhiên cải thiện điều này bằng cách phát hiện các dãy con đã được sắp xếp trước đó, sau đó trộn chúng lại, giúp giảm số lần chia nhỏ mảng.

Ý tưởng thuật toán:

1. Quét qua mảng và tìm các dãy con liên tiếp đã được sắp xếp (*runs*).
2. Trộn các dãy con này theo từng cặp cho đến khi mảng được sắp xếp hoàn toàn.

Chi tiết cài đặt:

```

#include <bits/stdc++.h>

int n = 8;
int a[] = {0, 1, 3, 5, 2, 4, 6, 8, 7};

// Hàm trộn hai đoạn đã sắp xếp trong mảng

```

```
void merge(int l, int m, int r) {
    int temp[r - l + 1];
    int i = l, j = m + 1, k = 0;

    while (i <= m && j <= r) {
        if (a[i] <= a[j]) temp[k++] = a[i++];
        else temp[k++] = a[j++];
    }

    while (i <= m) temp[k++] = a[i++];
    while (j <= r) temp[k++] = a[j++];

    for (int t = 0; t < k; ++t) a[l + t] = temp[t];
}

int main() {
    while (true) {
        std::vector<std::pair<int, int>> runs;
        int i = 1;

        // Tìm các dãy con đã sắp xếp
        while (i <= n) {
            int start = i;
            while (i < n && a[i] <= a[i + 1]) ++i;
            runs.push_back({start, i});
            ++i;
        }

        // Nếu chỉ còn một dãy con, mảng đã sắp xếp xong
        if (runs.size() == 1) break;

        // Trộn các dãy con theo từng cặp
        for (int j = 0; j < (int) runs.size(); j += 2) {
            if (j + 1 < runs.size()) {
                merge(runs[j].first, runs[j].second, runs[j + 1].second);
            }
        }
    }

    // In ra mảng sau khi được sắp xếp tăng dần
    for (int i = 1; i <= n; ++i) {
        std::cout << a[i] << " ";
    }
}
```

Độ phức tạp:

- **Thời gian:**
 - **Trường hợp tốt nhất:** Nếu mảng đã sắp xếp từ trước, thuật toán chỉ cần duyệt qua một lần mà không cần trộn, do đó độ phức tạp là $O(n)$.
 - **Trường hợp ngẫu nhiên và tệ nhất:** Giả sử mảng có r dãy con tự nhiên, mỗi lần trộn tốn $O(n)$ và cần khoảng $\log r$ lần trộn. Xét trường hợp số dãy con có thể gần bằng n , thuật toán sẽ có độ phức tạp thời gian là $O(n \log n)$.
- **Không gian:**
 - Do sử dụng mảng tạm để trộn các phần mảng, thuật toán có độ phức tạp không gian $O(n)$.

2.4.2 K-way Merge Sort (Sắp xếp trộn K chiều)

Trên thực tế, Merge Sort truyền thống có độ sâu đệ quy lớn do mỗi lần chia chỉ tách mảng thành hai phần, làm tăng số lần gọi đệ quy và ảnh hưởng đến hiệu suất. Để giảm độ sâu này, thay vì chia đôi, **K-Way Merge Sort** hay **Sắp xếp trộn K chiều** chia mảng thành k phần nhỏ hơn. Mỗi phần sau đó được sắp xếp đệ quy và cuối cùng hợp nhất bằng heap. Nhờ cách tiếp cận này, độ sâu đệ quy giảm từ $O(\log_2 n)$ xuống $O(\log_k n)$, giúp giảm lần gọi của đệ quy, và cải thiện hiệu suất một cách đáng kể.

Ý tưởng thuật toán:

1. Chia mảng đầu vào thành K phần nhỏ hơn.
2. Sắp xếp từng phần bằng cách sử dụng đệ quy K-way Merge Sort.
3. Trộn K mảng đã sắp xếp thành một mảng duy nhất bằng hàng đợi ưu tiên (*min-heap*).

Chi tiết cài đặt:

```
#include <bits/stdc++.h>

const int K = 3; // Số phần chia

int n = 9;
int a[] = {0, 9, 4, 7, 6, 2, 8, 3, 5, 1};

void mergeKSortedArrays(int left, int right, int k) {

    // Element = (giá trị, chỉ số mảng con, vị trí trong mảng con)
    using Element = std::tuple<int, int, int>;
    std::priority_queue<Element, std::vector<Element>, std::greater<Element>> pq;

    int size = right - left + 1;
```

```
std::vector<int> indices(k, 0);
int subArraySize = (size + K - 1) / K;

for (int i = 0; i < k; ++i) {
    int subLeft = left + i * subArraySize;
    if (subLeft <= right) {
        pq.emplace(a[subLeft], i, subLeft);
        indices[i] = subLeft + 1;
    }
}

int temp[size];
for(int i = 0; i < size; ++i) {
    auto [val, subArrayIndex, pos] = pq.top();
    pq.pop();
    temp[i] = val;

    if (indices[subArrayIndex] <= right
        && indices[subArrayIndex] < left + (subArrayIndex + 1) * subArraySize) {
        pq.emplace(a[indices[subArrayIndex]], subArrayIndex, indices[subArrayIndex]);
        ++indices[subArrayIndex];
    }
}

for (int i = 0; i < size; ++i) {
    a[left + i] = temp[i];
}
}

void kWayMergeSort(int left, int right) {
    // Nếu mảng rỗng hoặc chỉ còn một phần tử thì kết thúc.
    if (left >= right) return;

    int size = right - left + 1;

    // Kích thước của mỗi mảng con
    int subArraySize = (size + K - 1) / K;

    // Lưu các đoạn sau khi cắt thành K đoạn.
    std::vector<std::pair<int, int>> ranges;

    for (int i = 0; i < K && left + i * subArraySize <= right; ++i) {

        int subLeft = left + i * subArraySize;
        int subRight = std::min(subLeft + subArraySize - 1, right);
```

```

        ranges.push_back({subLeft, subRight});
        kWayMergeSort(subLeft, subRight);
    }

    mergeKSortedArrays(left, right, ranges.size());
}

int main() {

    // Gọi hàm đệ quy
    kWayMergeSort(1, n);

    // In ra mảng sau khi được sắp xếp tăng dần
    for (int i = 1; i <= n; ++i) {
        std::cout << a[i] << " ";
    }
    return 0;
}

```

Độ phức tạp:

- **Thời gian:**
 - **Trường hợp tốt nhất:** Nếu mảng đã được sắp xếp, thuật toán vẫn phải chia và trộn lại, nên độ phức tạp là $O(n \log_K n)$.
 - **Trường hợp ngẫu nhiên và tệ nhất:** Trong cả hai trường hợp này, thuật toán cần chia mảng và trộn lại, dẫn đến độ phức tạp $O(n \log_K n)$.
- **Không gian:**
 - Do sử dụng bộ nhớ bổ sung cho việc trộn K mảng, thuật toán có độ phức tạp không gian là $O(n)$.

2.5 Specific**2.5.1 Counting Sort (Sắp xếp đếm)**

Sau khi cùng tìm hiểu về các thuật toán dựa vào các thao tác cơ bản, ta sẽ đặt câu hỏi liệu có thể sắp xếp mà không cần phải so sánh hay không? Và câu trả lời là có! **Counting Sort** hay **sắp xếp đếm** là một thuật toán sắp xếp tuyến tính không dựa vào phép so sánh mà dựa trên việc đếm số lần xuất hiện của mỗi phần tử để xác định vị trí chính xác của chúng. Thuật toán này hoạt động dựa trên ý tưởng “xếp bỏ vào chuồng”: mỗi phần tử được đưa vào đúng vị trí dựa trên số lượng phần tử nhỏ hơn nó. Counting Sort hữu dụng vì có độ phức tạp $O(n + k)$, trong đó k là giá trị lớn nhất trong mảng, giúp nó hoạt động hiệu quả khi k không quá lớn so với n .

Ý tưởng thuật toán:

1. Tìm giá trị lớn nhất trong mảng để xác định phạm vi cần đếm.
2. Đếm số lần xuất hiện của từng phần tử.
3. Dùng mảng đếm để xác định vị trí chính xác của từng phần tử trong mảng đã sắp xếp.

Chi tiết cài đặt:

```
#include <bits/stdc++.h>

int n = 9;
int a[] = {0, 4, 2, 2, 8, 3, 3, 1, 7, 5};

int main() {
    int maxVal = *std::max_element(a + 1, a + n + 1);
    std::vector<int> count(maxVal + 1, 0);

    // Đếm số lần xuất hiện của mỗi phần tử
    for (int i = 1; i <= n; ++i) {
        count[a[i]]++;
    }

    // Xây dựng mảng đã sắp xếp
    int index = 1;
    for (int i = 0; i <= maxVal; ++i) {
        while (count[i]--) {
            a[index++] = i;
        }
    }

    // In ra mảng sau khi được sắp xếp tăng dần
    for (int i = 1; i <= n; ++i) {
        std::cout << a[i] << " ";
    }
    return 0;
}
```

Độ phức tạp:

- **Thời gian:**
 - mọi trường hợp, độ phức tạp thuật toán là $O(n + k)$, với k là giá trị lớn nhất trong mảng.
- **Không gian:**
 - Sử dụng mảng đếm có kích thước $O(k)$, nên độ phức tạp không gian là $O(n + k)$.

2.5.2 Radix Sort (Sắp xếp cơ số)

Counting Sort sắp xếp bằng cách đếm số lần xuất hiện của mỗi phần tử, sau đó đặt chúng vào vị trí đúng. Tuy nhiên, nó không hiệu quả khi dữ liệu có giá trị lớn vì cần mảng đếm kích thước $O(k)$, với k là giá trị lớn nhất. Ví dụ, nếu sắp xếp số trong khoảng 10^9 , ta cần mảng đếm rất lớn, gây lãng phí bộ nhớ.

Radix Sort hay **sắp xếp cơ số** giải quyết bằng cách sắp xếp theo từng chữ số từ hàng thấp đến cao và dùng Counting Sort làm bước trung gian. Nhờ vậy, nó hoạt động tốt ngay cả với số lớn hoặc chuỗi có độ dài ngắn.

Ý tưởng thuật toán:

1. Xác định chữ số lớn nhất trong mảng để biết số lượng chữ số tối đa.
2. Sử dụng Counting Sort để sắp xếp các số theo từng chữ số, bắt đầu từ hàng đơn vị đến hàng cao nhất.
3. Lặp lại quá trình cho đến khi toàn bộ mảng được sắp xếp.

Chi tiết cài đặt:

```
#include <bits/stdc++.h>

int n = 9;
int a[] = {0, 170, 45, 75, 90, 802, 24, 2, 66};

// Hàm Counting Sort theo chữ số `exp`
void countingSort(int a[], int n, int exp) {
    int output[n + 1], count[10] = {0};

    // Đếm số lần xuất hiện của từng chữ số
    for (int i = 1; i <= n; ++i) {
        count[(a[i] / exp) % 10]++;
    }

    // Tính vị trí thực tế của từng phần tử
    for (int i = 1; i < 10; ++i) {
        count[i] += count[i - 1];
    }

    // Xây dựng mảng kết quả
    for (int i = n; i >= 1; --i) {
        int digit = (a[i] / exp) % 10;
        output[count[digit]] = a[i];
        count[digit]--;
    }

    // Sao chép mảng kết quả về mảng gốc
```

```

        for (int i = 1; i <= n; ++i) {
            a[i] = output[i];
        }
    }

int main() {
    int maxVal = *std::max_element(a + 1, a + n + 1);

    // Sắp xếp theo từng chữ số, bắt đầu từ hàng đơn vị
    for (int exp = 1; maxVal / exp > 0; exp *= 10) {
        countingSort(a, n, exp);
    }

    // In ra mảng sau khi sắp xếp
    for (int i = 1; i <= n; ++i) {
        std::cout << a[i] << " ";
    }
    return 0;
}

```

Độ phức tạp:

- **Thời gian:**
 - Trong mọi trường hợp, độ phức tạp thuật toán là $O(d(n + k))$, với d là số chữ số lớn nhất và $k = 10$ cho hệ thập phân.
- **Không gian:**
 - Sử dụng mảng đếm và mảng trung gian có kích thước $O(n + k)$, nên độ phức tạp không gian là $O(n + k)$.

3 Tổng kết

Và như vậy, chúng ta đã cùng tìm hiểu về nhiều thuật toán sắp xếp khác nhau, từ những thuật toán cổ điển đến các phiên bản tối ưu hơn. Qua đó, chúng ta có thể thấy rằng việc tối ưu thuật toán giúp giảm thời gian thực thi, cải thiện hiệu suất và tiết kiệm tài nguyên. Chẳng hạn, K-Way Merge Sort giảm độ sâu đệ quy so với Merge Sort, giúp xử lý dữ liệu nhanh hơn. Radix Sort và Counting Sort khai thác đặc điểm dữ liệu để sắp xếp nhanh hơn so với các thuật toán dựa trên so sánh. Những cải tiến này đặc biệt quan trọng khi làm việc với dữ liệu lớn hoặc khi yêu cầu hiệu suất cao. Tuy nhiên, không phải lúc nào tối ưu hóa cũng đem lại lợi ích thực tế, vì một số thuật toán có thể tăng độ phức tạp mà không cải thiện đáng kể hiệu suất. Ví dụ, Shake Sort là một biến thể của Bubble Sort nhưng không làm giảm đáng kể thời gian thực thi, khiến nó tệ hơn khi được sử dụng trong thực tế.

Tóm lại, việc lựa chọn thuật toán phù hợp phụ thuộc vào nhiều yếu tố, không chỉ là độ phức tạp lý thuyết mà còn cả tính ứng dụng trong từng tình huống cụ thể.